Ders 12 Genişletilmiş Notu: Fonksiyon Çağrısında Tip Dönüşümü ve İteratif Algoritmaların Analizi: Temel İşlem ve T(n)

Ders 12 Genişletilmiş Notu: Fonksiyon Çağrısında Tip Dönüşümü ve İteratif Algoritmaların Analizi: Temel İşlem ve T(n)

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Doğrusal Sıralamadaki İki Sorun
- 2. Bir Öneri: Çift Yönlü Selection Sort
- 3. Bir Öneri: Yeni Diziyi Eleman Toplamı Kadar Açmak
- 4. Parantez, İlk Değer ve Girinti Hataları
- 5. Bu Kurallar Neden Var? Standartlar ve Tutorial'lar
- 6. Parametre Aktarım Mekanizmalarının Tekrarı
- 7. Fonksiyon Çağrısında Tip Dönüşümü
- 8. Java ile Karşılaştırma
- 9. Doğrusal Sıralamanın Sorunlarına Hocanın Önerdiği Çözümler
- 10. İteratif Algoritmaların Analizi: Amaç ve Kavramlar
- 11. Bütün Kodu Ayrıntılı Saymak
- 12. Temel İşlemle Saymak ve Forward Substitution
- 13. İç İçe Döngüler
- 14. Son Değer ile Karmaşıklık Farklı Şeylerdir
- 15. Ders Sonrası Kod Soruları
- 16. Dil Öğrenmek ve Öğrenmeyi Öğrenmek

1. Doğrusal Sıralamadaki İki Sorun

Dersin başında öğrenciler, geçen derste görülen iki doğrusal sıralama algoritmasına dair çözümlerini gösterdi. Hoca sorunun ne olduğunu birkaç kez netleştirmek zorunda kaldı, çünkü birçok öğrenci algoritmanın kendisini kodlamayı istenen şey sanmıştı. Algoritmayı kodlamak elbette tavsiye edilen bir çalışmaydı, ama asıl soru karşılaşılan problemlerin nasıl çözüleceğiydi. Bu problemler şunlardı:

- **Aynı değerde iki eleman:** Birinci algoritmada, dizide aynı değerden iki eleman varsa bir problem ortaya çıkıyordu.
- **Çok büyük değerler:** İkinci algoritmada, dizide çok büyük bir değer varsa yeni bir yere kopyalarken çok büyük bir dizi açmak gerekiyordu. Hoca, ikisinde de elemanlar çok büyükse bellek problemi olabileceğini söyledi.

Bir öğrenci önce `#define MAX` ile sabit boyut tanımlamayı sorun olarak gördü. Hoca bunun kastedilen şey olmadığını belirtti: Dizinin uzunluğunun n olması veya dinamik olmaması problem değildir. Problem, eşit değerler ve büyük değer aralığıdır.

2. Bir Öneri: Çift Yönlü Selection Sort

2.1 Algoritmanın Fikri

Bir öğrenci, doğrusal sıralamadan bağımsız olarak aklına gelen bir sıralama fikrini gösterdi. Her turda dizinin hem en küçük hem en büyük elemanı bulunur, ikisi de yerine konur ve aralık iki taraftan daralır. Öğrencinin anlattığı adımlar şöyledir:

1. Dış döngü, eleman sayısının yarısı kadar tur döner.
2. Minimumun yeri baştan, maksimumun yeri sondan başlar. Tur ilerledikçe minimumun yeri artar, maksimumun yeri azalır.
3. İç döngü aradaki elemanları gezer. Eleman maksimumdan büyükse maksimum güncellenir, değilse minimuma bakılır.
4. Seçilen minimum, maksimumdan büyük olursa yanlış sonuç çıktığı için önce bu iki eleman yer değiştirilir.
5. Minimum ve maksimum, yardımcı bir `temp` değişkeniyle yerlerine yazılır.
6. Öğrenci, kodunda bir sayaç da tutar. Bunu kontrol etmediğinde çift eleman sayılı dizilerde iki kez değişim yapıldığını ve dizinin tam sıralanmadığını söylemiştir.

Hoca bunun çift yönlü selection sort olduğunu söyledi. Selection sort en küçük değeri bulup başa yazar. Burada ayrıca en büyük değer bulunup sona yazılır. Bu yüzden derste beklenen çözümle ilgisi yoktur, ama güzel bir yaklaşımdır. Tek bir dizi üzerinde yaklaşık n^2 defa işlem yapılır.

2.2 Quick Sort ile Bir Bağlantı

Hoca bu noktada Quick Sort'u andı. Quick Sort özyinelemeli algoritmalarla birlikte anlatılacaktır. Algoritma Analizi ve Veri Yapıları derslerinde de farklı bakış açılarıyla ele alınır. Hocaya göre Quick Sort'un hızlı olmasının en önemli sebeplerinden biri, tek bir iterasyonda iki elemanın yer değiştirebilmesidir. Bubble sort, selection sort ve insertion sort'ta ise işlem bir eleman üzerinden yapılır.

> **Ders sonrası not:** Hoca Quick Sort'un hızının önemli bir sebebini tek iterasyonda iki elemanın yer değiştirmesi olarak anlattı. Asıl sebep büyük olasılıkla diziyi pivot etrafında iki parçaya bölüp parçaları ayrı ayrı sıralamasıdır. Bu sayede ortalama karmaşıklık büyük olasılıkla $n \log n$ olur. Bir turda iki eleman yerleştirmek tek başına bunu sağlamaz. Bu bölümdeki çift yönlü selection sort da her turda iki eleman yerleştirir, ama hâlâ n^2 düzeyindedir.

2.3 Tahtada Adım Sayısı

Hoca, kodu açtırıp kabaca bir analiz yaptı. Dış döngünün değişkeni i, 0'dan $n/2$'ye birer birer gider. Klasik selection sort'ta bu sınır n'dir, yani bir iyileşme buradan gelir. İç döngü $i+1$'den başlar ve $n-i-1$'e kadar gider. Bu bir terim sayısı sorusudur:

$$(n-i-1)-(i+1)+1 = n-2i-1$$

Derste önce $n-2$ olup olmadığı soruldu, hoca $n-2i-1$ olduğunu teyit etti. Her adıma bir birim dersek toplam şudur:

$$\sum_{i=0}^{n/2-1} (n-2i-1) = \frac{n}{2}(n-1) - 2\sum_{i=0}^{n/2-1} i$$

Hoca, $n/2$ tane $n-1$ teriminden $\frac{n}{2}(n-1)$ geldiğini, bundan 2 çarpı bir toplamın çıkarılacağını gösterdi. Sonucun $\frac{n(n+1)}{2}$'den daha düşük bir sayı olacağını söyledi. Bu yüzden süre karşılaştırmasında klasik selection sort'tan daha hızlı sonuç beklenebilir.

Kapalı biçim şöyle tamamlanır (ara adım eklendi). n çift iken $\sum_{i=0}^{n/2-1} i = \frac{n}{2}\left(\frac{n}{2}-1\right)\frac{1}{2}$ olur. Yerine yazınca:

$$\frac{n}{2}(n-1) - \frac{n}{2}\left(\frac{n}{2}-1\right) = \frac{n}{2}\cdot\frac{n}{2} = \frac{n^2}{4}$$

Öğrencinin örnek verdiği 6 elemanlı dizide iç döngü sırayla 5, 3 ve 1 adım döner. Toplam $9 = 6^2/4$ olur (python ile doğrulandı).

> **Ders sonrası not:** Hoca toplamı tahtada 0'dan $n/2$'ye kadar yazıp $\frac{n}{2}\left(\frac{n}{2}+1\right)\frac{1}{2}$ dedi. Dış döngü $n/2$ tur döndüğü için üst sınır büyük olasılıkla $n/2-1$ olmalıdır. Bu durumda sonuç yukarıdaki $n^2/4$'tür (python ile doğrulandı). Kabaca n^2 düzeyinde olduğu için hocanın vardığı sonuç değişmez.

2.4 Süre Karşılaştırması Nasıl Yapılır?

Hoca, süre ölçmek için şu deneyi önerdi:

1. A dizisi rastgele sayılarla doldurulur (örneğin 1 milyon uzunluklu).
2. A doldurulurken aynı değerlerle B dizisi de doldurulur. Çünkü iki algoritma aynı şartlarda, aynı elemanlar ve aynı dağılımla karşılaştırılmalıdır.
3. Kodda `printf` gibi hiçbir şey olmaz.
4. Önce selection sort çalıştırılır. Başına ve sonuna birer zamanlayıcı konup aradaki fark alınır.
5. Aynısı yeni yazılan kod için yapılır.
6. Deney 1 milyon, 10 milyon ve 100 milyon gibi boyutlarda tekrarlanır.

Hoca bir de çekince ekledi. Kodda çok fazla atama vardır ve her adımda iki kontrol yapılabilir. Biz o işlemi bir birim kabul ettik, ama iki kabul edersek sonuç normal selection sort'la başa baş çıkabilir. n^2'nin içindeki alt terimler (n^2+n gibi) süreyi etkileyen diğer parametrelerdir. Bunu görmek için kodu satır satır analiz etmek gerekir. Dersin ikinci yarısı da bu konudur.

> **Ders sonrası not:** Hocanın başa baş uyarısı formülle de görülür. İç döngü adımı $n^2/4$'tür. Her adımda iki karşılaştırma yapılırsa toplam $n^2/2$ olur. Klasik selection sort büyük olasılıkla $n(n-1)/2$ karşılaştırma yapar, yani ikisi neredeyse aynıdır (python ile doğrulandı). Öğrencinin anlattığı "değilse minimuma bak" yapısında bir adımdaki karşılaştırma sayısı veriye göre 1 veya 2 olur. Ayrıca önerilen 100 milyon elemanda iç döngü $n^2/4 = 2{,}5\cdot 10^{15}$ adım tutar. Bu boyutta n^2'lik bir sıralamanın ölçümü büyük olasılıkla pratikte bitmez, deney küçük boyutlardan başlatılmalıdır.

3. Bir Öneri: Yeni Diziyi Eleman Toplamı Kadar Açmak

Başka bir öğrencinin önerisi, büyük değer sorununa yönelikti. `#define MAX` ile gereğinden büyük bir boyut vermek yerine, girdiler okunurken elemanlar `toplam` adlı bir değişkende toplanır ve ikinci dizi bu toplama göre açılır. Örneğin 1, 2, 3, 4, 5 girilince 15'lik yer açılır. Öğrenci, maksimumu bulmak için ayrı bir `for` döngüsü eklemek yerine girdi okunurken toplamayı daha az işlem olarak görmüştü.

Hoca bu çözümü birkaç açıdan inceledi. Bunlar bu dersin en öğretici hata listesidir:

- **Sorunu çözmüyor:** Dizide 1 milyon varsa toplam da en az o kadar olur ve yine çok büyük bir dizi açmak gerekir.
- **Küçük değerlerde de israf:** 1, 2, 3, 4, 5 için 5 yer yeterliyken 15 yer açılır. 1, 2, 3 için 3 yer yeterliyken 6 yer açılır. Toplam, en büyük elemandan daima büyüktür.
- **Negatif elemanlar:** Elemanlar negatif olursa dizinin boyutu da negatif çıkar.
- **Dosya uzantısı:** Kod `.cpp` olarak kaydedilmiştir. Bu "C görünümlü C++ kodu"dur.
- **Değişkene bağlı boyut:** `int A[n]` ve `int B[toplam]` gibi bir tanım C'de hatadır. Kod `.cpp` olduğu için hata alınmıyor olabilir. Hoca, `.c` olarak kaydedilip n yerine 1 milyon girilirse büyük olasılıkla segmentation fault alınacağını, bellekte yer ayrılamayacağını söyledi. Şu ana kadarki bilgilerle boyuta 10, 20 gibi bir sabit veya `#define` ile tanımlı bir ad yazılmalıdır.
- **İlk değer verilmemiş toplam:** Kümülatif toplam yapılan değişkene ilk değer verilmezse, bellekte o değişkenin son değeri neyse ondan devam edilir. Sıfırsa sıfırdan, çok büyük bir sayıysa ondan devam eder ve program hatalı sonuç verir.

- **Ortada değişken tanımı:** 14. satırda bir dizi tanımlanmıştır. C'de değişkenler başta tanımlanır, arada tanımlanmaz.

  Hocanın vurgusu: Kodlarınızı `.cpp` olarak değil, `.c` olarak kaydedin. Değişkenleri başta tanımlayın, dizi boyutunu sabit veya `#define` ile verin, kümülatif toplam yapılan değişkene ilk değer verin. Sonraki yılların algoritma derslerinde bu hatalar çok pahalıya gelir. Dosya adlarında da Türkçe karakter kullanmayın.

  Ders sonrası not: Hoca değişkene bağlı boyutu ve ortada değişken tanımını C'de hata olarak anlattı. Bu kural büyük olasılıkla C89 standardına aittir. C99'dan itibaren ikisi de dilde vardır (değişken uzunluklu dizi, VLA). Dersin ve değerlendirmenin kuralı bu olduğu için hocanın yazım biçimine uyulmalıdır. Hocanın uyarısı VLA için de büyük olasılıkla geçerlidir, çünkü bu diziler yığında oluşur ve büyük boyutta program çöker.

Soru: Maksimumdan bir fazla uzunluk da tanımlanabilirdi. Ama bunun için bir `for` döngüsü daha eklemek ek karmaşıklık getirmez mi?

Cevap: Birbirinden bağımsız, iç içe olmayan iki `for` döngüsü yazmakla tek döngü yazmak arasında algoritmik ve asimptotik olarak ciddi bir fark yoktur. İç içe iki döngü, sınırlarına göre, n^2'lik karmaşıklığa götürebilir. Ayrı iki döngü ise $2n$'e götürür ve $2n$, n gibi kabul edilir, baştaki katsayı silinir. n^2 ile $2n$ arasındaki fark ise çok ciddidir.

4. Parantez, İlk Değer ve Girinti Hataları

Bir öğrencinin paylaştığı kodda şu hatalar vardı:

- Fonksiyon `main`'den önce tanımlanmış, ama kapanış parantezi unutulmuştu. Bu yüzden derleyici `is not allowed here` diyordu.
- Derleyici, `j` değişkeni için ilk değer verilmediği uyarısını verdi (`variable j uninitialized`). Hoca, bir önceki koddaki `toplam` değişkeninde aynı hatanın neden görülmediğini açıkladı: Bazı derleyiciler bunu bildirmez. Öğrencinin Mac'teki derleyicisi bildirir. Bu, C açısından aslında bir uyarıdır.
- Derleyici, tanımsız bir değişken için `use of undeclared identifier` hatası verdi.

Bu sırada kod biçimlendirmesi konuşuldu. Öğrencinin "tidy up" diye sorduğu şey, hocanın ifadesiyle code formatting ve daha çok girinti (indentation) anlamındaydı: `for` gibi `if` gibi blokların içi bir tab sağa yazılır. `printf` ve `scanf` satırlarından sonra içeri girmeye gerek yoktur, bu yalnızca bloklar içindir.

5. Bu Kurallar Neden Var? Standartlar ve Tutorial'lar

Hoca, derste söylediği yazım kurallarının kendi uydurduğu bir stil olmadığını vurguladı. Değişkenlerin nerede tanımlanacağı, dizi boyutunun nasıl verileceği, fonksiyona değer aktarımının nasıl yapılacağı standarttır. Bölümdeki diğer hocalar da aynısını ister. Hoca, Veri Yapıları, Yapısal Programlama ve algoritma derslerine de katıldığı için istenenin bu olduğunu söyledi. Her kuralın bir gerekçesi vardır. Örneğin "global değişken kullanmayın" denmesinin işletim sistemlerinde race condition ile ilgili bir karşılığı vardır.

> **Ek bilgi (derste anlatılmadı):** Race condition, büyük olasılıkla aynı veriye birden fazla iş parçacığının aynı anda erişmesi ve sonucun bu erişimlerin sırasına bağlı olması durumudur. Global bir değişkene her fonksiyon erişebildiği için böyle bir paylaşım kolayca oluşabilir. Yerel değişkenler ve parametrelerle çalışmak bu riski büyük olasılıkla azaltır.

Tutorial siteleri konusunda hoca bir anı anlattı. Yaklaşık 2018'de, yapay sinir ağları üzerine ders ve proje çalışmaları sürerken bir firma, fakültede derin öğrenme semineri vermişti. Hoca, bir saatlik konuşmada teknik ve kitabi açıdan 5-6 yanlış bilgi saymıştı. Seminere giden öğrenciler ise "uzun süredir öğrenemediğimizi bir saatte öğrendik" diye dönmüştü. Hocaya göre sunum güzeldi, ama o an doğru gibi görünen bir bilgi genele vurulunca yanlış oluyordu. Bütün problemler o şekilde çözülemezdi.

Hocanın vurgusu: Tutorial'larda özel bir durum için doğru görünen, ama genele bakıldığında yanlış olan çok şey vardır. Tutorial'lardan faydalanın, ama derste öğrendiklerinize ek olarak kullanın. Bilgisayar sizin için bir araç değil, amaçtır.

6. Parametre Aktarım Mekanizmalarının Tekrarı

Hoca C'nin temel karakterini yeniden hatırlattı: C'de hemen her şeyi kendiniz düşünüp tasarlamanız gerekir. Birçok kontrol size bırakılmıştır. Anahtar kelimesi ve kuralı az olduğu için daha hızlıdır, ama güvenli değildir. Hocanın ifadesiyle C "sen kendi başının çaresine bak" der.

Fonksiyonlara parametre geçişinde iki mekanizma vardı: call by value ve call by reference. Call by reference iki amaçla kullanılır:

- Değişkenlerin değerini değiştirmek (matris güncellemek de buna girer).
- Birden fazla geri dönüş değeri elde etmek.

Bir `return` en fazla bir şey döndürür. Bu `int`, `int` pointer, karakter, karakter pointer veya `int` çift pointer olabilir. Ama `a` ve `b` gibi iki tam sayı döndürülemez, ya `a` ya `b` döner. Bunu aşmak için adres gönderilir. Hocanın örneğinde $x = 5$, $y = 8$ iken `g` fonksiyonuna iki adres gider:

```c
#include <stdio.h>

void g(int *t, int *l);

int main(void)
{
    int x = 5, y = 8;

    g(&x, &y);
    printf("x=%d y=%d\n", x, y);
    return 0;
}

void g(int *t, int *l)
{
    *t = 0;
    *l = 0;
}
```

`*t = 0` ve `*l = 0`, "x ve y'nin değerlerini 0 yap" demektir. Program `x=0 y=0` yazar (`gcc -Wall -std=c89 -pedantic` ile uyarısız derlendi). Hoca, adresi gönderilecek değişkenlere ilk değer vermenin zorunlu olmadığını da ekledi, çünkü değer fonksiyonda atanabilir. Call by value bunu yapamaz. Call by reference hem içeriği değiştirir hem birden fazla değer dönüşünü mümkün kılar.

Aynı mekanizmayı `scanf` de kullanır. `scanf("%d %c", &x, &c);` yazıldığında iki değişkenin adresi gider ve bu fonksiyondan iki değer değişir. Yani bir anlamda iki değer döner. Bir `%d` daha eklenip `&y` de verilirse üç değişken değişir, çünkü `scanf`'in tanımı çok sayıda parametre almaya uygundur.

7. Fonksiyon Çağrısında Tip Dönüşümü

Bu iki mekanizmanın yanında, değer aktarımında bir de tip dönüşümü dikkate alınmalıdır. Hoca bunu C'de çok dikkat edilmesi gereken, ama göz ardı edilen bir problem olarak tanımladı. Tip hiyerarşisini daha önce bir slaytta göstermişti.

Önce sabitlerin türü soruldu. `3` tam sayı (`int`) sabitidir. `3.0` kayan noktalıdır. `'3'` bir karakterdir. `"3"` ise bir karakter katarıdır (string). Parametre aktarımında sabitin türüne göre bir dönüşüm yapılabilir.

7.1 Deneyler

Hoca şu iki fonksiyonu yazıp farklı çağrılarla çalıştırdı. İkisinin gövdesi aynı, parametre türü farklıdır:

```
#include <stdio.h>

void f(int t);
void h(char t);

int main(void)
{
    f(3);       printf("\n");
    h(3);       printf("\n");
    f('A');     printf("\n");
    f(5.40);    printf("\n");
    h(256);     printf("\n");
    h(257);     printf("\n");
    return 0;
}

void f(int t)
{
    printf("%d--%c", t, t);
}

void h(char t)
{
    printf("%d--%c", t, t);
}
```

`f(3)`: 3 tam sayıdır, parametre de tam sayıdır. Call by value gereği bellekte `t` için yeni bir tam sayı alanı açılır ve 3 oraya birebir kopyalanır. Ekranda önce 3, sonra `%c` karşılığında bir kalp işareti görüldü.

`h(3)`: Bu kez 3 `int`, parametre `char`'dır. 3, `char`'a dönüştürülür ve sonuç bu dönüşüme göre görülür. Çıktı yine 3 ve kalp işaretidir. Hoca, `printf` içinde `%c` ile yapılan gizli dönüşümün aynısının fonksiyon çağrısında da olduğunu vurguladı.

> **Ders sonrası not:** Hoca, 3'ün ASCII kodunun (48'den başladığı için yaklaşık 51) karaktere dönüştürüldüğünü söyledi. Doğrusu büyük olasılıkla şudur: 51, `'3'` karakterinin kodudur. Burada gönderilen şey 3 sayısıdır, dönüşümden sonra da `t` 3 olarak kalır. `%c` ile 3 kodlu kontrol karakteri basılır ve Windows konsolu onu büyük olasılıkla kalp işareti olarak gösterir. Linux terminalinde aynı karakter görünmez (gcc ile çıktı olarak 3 ve 3 kodlu bayt doğrulandı).

`f('A')`: Gönderilen karakter `f`'e gider. `'A'`'nın karşılığı 65'tir. `%d` 65 yazar. `%c` ise 65'i tekrar karaktere çevirip A yazar. Çıktı `65--A` olur.

String göndermek: Sonra `f`'e `"A"` gibi bir string gönderildi. String doğrudan tam sayıya dönüştürülemez. Ama C derleme zamanında hata vermez. Hocanın ifadesiyle C "bu işle ben ilgilenmem, fonksiyonu kendin yazdıysan nasıl değer göndereceğini kendin bilmen lazım" der. Olan şudur: String'in bellekte bir yeri vardır. Bu adresin değeri alınıp `t`'ye kopyalanmaya çalışılır. Ekranda anlamsız bir sayı çıkar. Hoca, karakteri değiştirip aynısını gösterdi ve bir diziyi fonksiyona aktarırken ilk elemanının adresinin gittiğini hatırlattı. String de bir karakter dizisidir.

> **Ders sonrası not:** Hoca, adresin karakter karşılığı olmadığı için `%c` yerine bir şey yazılmadığını söyledi, çünkü ASCII'nin 255'e kadar olduğunu belirtti. Standart ASCII büyük olasılıkla 0-127 arasıdır, 128-255 genişletilmiş kod sayfalarına aittir. Ayrıca `%c`, verilen sayıyı büyük olasılıkla `unsigned char`'a çevirir, yani sayının son 8 bitini basar. Görünür bir şey çıkmaması bu baytın yazdırılamayan bir karakter olmasından kaynaklanır. Bir de `'A'` C'de büyük olasılıkla zaten `int` türündedir (`sizeof('A')` burada 4 çıktı), yani bu çağrıda fiilen bir dönüşüm yoktur.

7.2 Büyük Tipi Küçük Tipe Göndermek

Hoca `f`'e önce 3.4, sonra 4.4 gönderdi. Bu, büyük bir veri tipini (`double` veya `float`) küçük bir tipe dönüştürmektir. Program hiçbir hata vermeden çalıştı, hatta ekranda -1 çıktı. 5.4 ile de 5 yerine -5 görüldü. Hoca 5.40'tan normalde 5 görmemiz gerektiğini, bunun neden böyle çıktığını bilmediğini ve araştırılması gerektiğini söyledi. Bilmemiz gereken asıl şey, hata alınmamasıdır.

Hoca ardından `char`'ın en büyük değerini aşan değerleri denedi. `h(256)` için derleyici yalnızca uyarı verdi ve ekranda 0 çıktı. `h(257)` için 1 çıktı. Hoca bunun 256'ya göre mod almak gibi göründüğünü söyledi:

$$256 \bmod 256 = 0, \qquad 257 \bmod 256 = 1$$

Prototip uyarıları da bu sırada görüldü. Hoca fonksiyon prototiplerini yukarı yazınca bir kısım uyarı kayboldu. Geriye iki uyarı kaldı: string'in adres olarak gönderilmesi (pointer'ın tam sayıya verilmesi) ve `h`'ye verilen değerin taşması (overflow). Bunların ikisi de programın çalışmasına engel değildir.

> **Ders sonrası not:** -1 ve -5'in sebebi büyük olasılıkla prototipin eksik olmasıdır. Prototip yokken `5.40` `double` olarak gönderilir, fonksiyon ise bir `int` okur. Sonuç tanımsız davranıştır. Burada `gcc -std=c89` ile prototipsiz denemede `conflicting types` uyarısı çıktı ve `f(4.4)`, `f(5.40)` anlamsız büyük bir negatif sayı yazdı. Prototip eklenince `5.40` önce `int`'e dönüştürülür, kesirli kısım atılır ve `f(5.40)` 5 yazar (gcc ile doğrulandı). Yani fonksiyon prototiplerini yazmanın somut bir gerekçesi budur.
>
> **Ders sonrası not:** Hoca `char` için "8 bit, en büyük değer 256" dedi. 8 bit büyük olasılıkla 256 farklı değer demektir. En büyük değer `signed char` için 127, `unsigned char` için 255'tir. İşaretli tipe sığmayan değerin dönüşümü büyük olasılıkla derleyiciye bırakılmıştır. GCC'de `h(256)` 0, `h(257)` 1 verir ve `overflow in conversion from 'int' to 'char'` uyarısı çıkar (gcc ile doğrulandı). Hoca ayrıca küçük tipten büyüğe dönüşümde de veri kaybı olabileceğini söyledi. Değer kaybı asıl olarak büyük olasılıkla büyükten küçüğe dönüşümde olur.
>
> **Hocanın vurgusu:** Tip dönüşümü çok yapılan bir hatadır. C burada derleme veya çalışma zamanı hatası vermez. Doğru tipte değer göndermek sizin sorumluluğunuzdur.

8. Java ile Karşılaştırma

Hoca Java dersi de verdiği için dili karşılaştırmalı anlatmayı tercih etti. Java'da aynı adlı fonksiyon, parametreleri sayı, sıra veya tip açısından farklı olmak üzere defalarca tanımlanabilir. Buna overloading denir. C'de böyle bir şey yoktur.

Tip dönüşümünde de iki dil ayrılır. Java, çağrıda büyük bir tipin küçük bir tipe dönüştürülmesine izin vermez. C ise doğrudan izin verir. Hocanın gösterdiği Java örneğinde farklı türlerde değişkenler tanımlanmış, aynı adlı birkaç fonksiyon çağrılmıştı. İki tür derleme zamanı hatası görüldü:

- **Lossy conversion:** Parametreleri `int, char, int` olan fonksiyon uygulanabilir değildir, çünkü `long`'dan `char`'a dönüşüm gerekir. `int, short, long` olanda da `long`'dan `short`'a dönüşüm gerekir. Bunların ikisine de derleyici izin vermez.
- **Ambiguity:** Çağrı aynı anda iki tanıma uyuyorsa, örneğin hem `char, float, double` hem `int, float` ile tanımlı fonksiyona gidebiliyorsa, derleyici hangisine gideceğini bilemediğini söyler.

Bu yüzden Java'da kod yazmak biraz daha kolaydır. C'de bu kontrolleri kendiniz yaparsınız, doğru düzgün çağırmazsanız farklı sonuçlar alırsınız.

> **Ek bilgi (derste anlatılmadı):** Java'nın izin vermediği şey büyük olasılıkla örtük (implicit) daraltmadır. Programcı `(char) uzunSayi` gibi açık bir cast yazarsa dönüşüm büyük olasılıkla derlenir. Yani Java, kayıp ihtimalini programcıya açıkça yazdırır.

Hocanın çok duyduğu bir cümle de buna bağlıdır: “Hocam kod yazdım, herhalde bilgisayarda bir sıkıntı var, yanlış sonuç veriyor.” Hocaya göre o yanlış sonuç değildir, büyük ihtimalle bir yerde ufak bir yazım hatası vardır.

Hoca bu bölümü, bu dönem için C’de stringler dışında çok fazla konu kalmadığını söyleyerek kapattı. Diziler, matrisler, fonksiyonlar ve pointer’larla sayısal analizdeki ve bu dersteki bütün algoritmalar rahatça yazılabilir.

9. Doğrusal Sıralamanın Sorunlarına Hocanın Önerdiği Çözümler

Aradan sonra hoca, baştaki soruya kendisinin beklediği yaklaşımları anlattı. Herkesin kabul edeceği tek bir çözüm yoktur, kişisel alternatifler vardır. Çünkü bu algoritma bellek-zaman ödünleşimini (trade-off) gösterir: Zamandan kazanırken bellekten kaybedebilir veya doğruluktan ödün verebilirsiniz. Hashing mantığıyla da yaklaşılabilir. Hoca matrisle bir çözüm beklediğini söyledi.

9.1 Eşit Elemanlar İçin: Frekans Saklamak

Hoca dizi olarak 6, 3, 8, 2, 5, 9 değerlerini yazdı, sonra 6’yı iki tane yaptı. En büyük değer 9 olduğu için 0’dan 9’a kadar 10 elemanlık bir yer açılır. Önce değer ve frekans için iki sütunlu bir yapı çizmeye başladı, ama sonra ikinci boyuta gerek olmadığını fark etti: Değer zaten indistir, hücrede yalnızca frekans saklanır. Bütün hücreler başta 0’dır.

Adımlar şöyledir (6, 3, 8, 2, 5, 9, 6 dizisi için, python ile doğrulandı):

1. Dizi baştan sona gezilir. Her değerin hücresindeki frekans bir artırılır. Sonunda 2, 3, 5, 8 ve 9 hücrelerinde 1, 6 hücresinde 2 yazar. Diğerleri 0’dır.
2. Yeni dizi için 0. hücreden başlanır. 0 tane varsa hiçbir şey yazılmaz.
3. Her değer, frekansı kadar yeni diziye yazılır. Önce 2, sonra 3, sonra 5, sonra iki tane 6, sonra 8 ve 9.

Sonuç 2, 3, 5, 6, 6, 8, 9’dur. Böylece aynı değerden birden fazla eleman olması sorun olmaktan çıkar.

9.2 Bellek İçin: Değer-Frekans Tablosu

İkinci yaklaşımda yalnızca dizide geçen değerler, iki sütunlu (değer, frekans) bir yapıda saklanır. Yapı, insertion mantığıyla sıralı tutulur. Yukarıdaki diziyle:

Okunan eleman	Tablonun yeni hâli (değer, frekans)
6	(6, 1)
3	(3, 1), (6, 1)
8	(3, 1), (6, 1), (8, 1)
2	(2, 1), (3, 1), (6, 1), (8, 1)

Yeni gelen 3 için 6 ve frekansı bir aşağı kaydırılır ve 3 üste yazılır. 2 geldiğinde 8, 6 ve 3 aşağı kayar. Tabloda zaten var olan bir değer gelirse yalnızca frekansı artar. Hocanın örneğinde ikinci 6 geldiğinde 6’nın frekansı 2 olur. Sonunda tablo baştan sona okunur ve her değer frekansı kadar diziye yazılır.

> **Ders sonrası not:** Hoca insertion adımında “3’ün değeri 6’dan büyük olduğu için” dedi. Doğrusu 3’ün 6’dan küçük olmasıdır. Tablo küçükten büyüğe tutulduğu için 3, 6’nın üstüne geçer.

Hoca bellek farkını şöyle gösterdi. Bu yöntemde en büyük elemana kadar yer açmaya gerek yoktur. Dizide 100 eleman varsa tablo en fazla 100 satırdır. Bir eleman birden fazla geçiyorsa daha da azdır. Yine bir bellek kaybı vardır, ama çok daha az yer işgal edilir. Daha iyi çözümler linked list ve hash mantıklarıyla yapılır. Bunlar sonraki dönemlerde Veri Yapıları ve Algoritma Analizi derslerinde görülecektir.

Soru: İndisleri kullanmak yalnızca doğal sayıları modelleyebiliyor mu?

Cevap: Evet, daha farklı sayılar varsa ve indisler işe yaramıyorsa, iki boyutlu bir yapı veya iki dizi kullanılabilir.

Soru: Bellek tarafında bir çözüm bulamadım. Ne yapılabilir?

Cevap: Mantık, örneğin 0'dan 100'e kadar 100 yer açıp yalnızca birkaçını kullanmak yerine, belleği daha iyi organize etmektir. Bunun verimli yollarını linked list konusunda gelecek dönem Yapısal Programlama dersinde göreceksiniz. Hepsi bu dersin konusu değildir. Bu konuya bir ucundan başlanınca çok ileri seviyelere gidilebilir.

10. İteratif Algoritmaların Analizi: Amaç ve Kavramlar

Dersin ikinci büyük konusu, kod parçalarının çalışma adımlarını saymaktır. Hoca önce neden bu işe girdiğimizi anlattı. Bir sıralama algoritmasına "n^2'lik" diyoruz, ikili aramaya "$\log n$" diyoruz. Peki bu n^2 veya $\log n$ nereden çıkıyor? Bunlar kodun analiz edilmesinden çıkıyor. Az önce çift yönlü selection sort'ta yapılan şey de buydu: koddaki toplam çalışma adımını (number of steps) saymak ve buradan n değişkenine bağlı bir fonksiyon üretmek.

Problemler genellikle fonksiyonlarla modüler hale getirilir. Fonksiyonlar iki türlüdür:

- **İteratif fonksiyonlar:** Yukarıdan aşağıya doğru çalışır, arada bir döngü varsa o döngü yapılır.
- **Recursive fonksiyonlar:** Fonksiyon, döngü kullanmadan kendisini tekrar eder. Bu ders işlenmedi, daha sonra görülecek.

İteratif fonksiyonları analiz ederken bulacağımız şey $T(n)$'dir. Bu fonksiyon, n sonsuza giderken programın nasıl çalıştığı, nereye doğru gittiği hakkında bilgi verir. Bir A algoritması için $T_A(n)$, bir B algoritması için $T_B(n)$ bulup ikisini karşılaştırırız. Aynısı bellek için de yapılabilir ($M_A(n)$ ve $M_B(n)$), ama bu ders kapsamında asıl üzerinde durulacak olan çalışma zamanıdır. Bu konu hem algoritmayı analiz etmeye hem de geliştirmeye yardım eder.

Hocanın örneği $n!$ hesabıydı: $n \cdot (n-1) \cdot (n-2) \cdots 1$. Burada yapılan işlem çarpmadır ve genel olarak n defa yapılır, $n-1$ demeye gerek yoktur. Buradan iki önemli kavram çıkar:

- **Temel işlem (basic operation):** Bütün kodu analiz etmek yerine, koddaki temel işlemi analiz etmek yeterlidir.
- **İterasyon sayısı (number of iterations):** Bu temel işlemin kaç defa tekrar edildiği.

10.1 Neler Sayılır, Neler Sayılmaz?

Analizde değişken tanımları dikkate alınmaz. `scanf` ve `printf` gibi fonksiyonlar da dikkate alınmaz. İlk değer verme, karşılaştırma, artırma gibi operatör içeren işlemler dikkate alınır. Bunlar tek operandlı (unary), iki operandlı (binary) ve tek örneği olan üç operandlı (ternary) operatörlerdir. Çoğunun doğrudan makine kodu karşılığı vardır. Örneğin karşılaştırma makine kodunda genellikle `CMP` ile veya çıkarma (`SUB`) ile yapılır. Hepsine bir birim maliyet (one unit) verilir ve n'e bağlı toplam maliyet kabaca bulunmaya çalışılır.

11. Bütün Kodu Ayrıntılı Saymak

Hocanın ilk kodu şudur. n'in `scanf` ile alındığı kabul edildi:

```c
#include <stdio.h>

int main(void)
{
    int i, n, T;

    scanf("%d", &n);
    T = 0;
    for (i = 1; i <= n; i++)
        T = T + 1;
    printf("%d", T);
```

```
    return 0;
}
```

Temel işlem kullanılmadan bütün kodu analiz edersek şöyle sayarız:

İşlem	Kaç kez?
`T = 0`	1
`i = 1`	1
`i <= n`	$n+1$
`T + 1`	n
`T = ...`	n
`i++` (hocanın sayımı)	$n+1$

Hoca karşılaştırmayı önce n yazdı, sonra düzeltti. Sebep `for` döngüsünün çalışma sırasıdır. `i = 1` bir kez yapılır. Sonra koşul kontrol edilir, doğruysa gövde çalışır, sonra `i++` yapılır ve tekrar kontrole gidilir. $n = 10$ girildiğinde son olarak `i = 10` için kontrol yapılır, gövde çalışır, `i` 11 olur, `i = 11` için kontrol yapılır ve döngüden çıkılır. Toplam $n + 1$ kontrol vardır.

Hoca bunları toplayıp $T(n) = 4n + 4$ buldu.

> **Ders sonrası not:** Hoca `i++` için de $n + 1$ dedi. `i++` büyük olasılıkla gövdeden sonra çalıştığı için n kez yapılır: $n = 10$ iken `i` 1'den 11'e 10 artışla çıkar. Doğru toplam $4n + 3$'tür ($n = 10$ için 43, python ile sayılarak doğrulandı). Hocanın vardığı sonuç olan $T(n) \approx n$ değişmez.

Bütün bu birimler kabul edilmiş maliyetlerdir. Süreyi saniye cinsinden düşünürsek toplama, karşılaştırma ve atama komutları makine seviyesinde farklı sürelerde çalışır. Biz hepsine bir birim dedik.

Sonra $T(n) = 4n + 4$ için şunu deriz: Sabit sayılar atılır, katsayılar atılır, en büyük dereceli terim alınır. Yani kabaca $T(n) \approx n$. Eğer $T(n) = 4n^2+4n-3$ çıksaydı n^2 derdik. İki algoritmayı karşılaştırırken bunu kullanacağız.

Peki detaylı analiz ne için gerekir? Bilgisayar mühendisi hangi işlemin maliyet gerektirdiğini bilmelidir: Bir tanım sıfırdır, şurası bir birimdir. Bunlar ayrıntılı sayılabilir, ama işin kısa yolu temel işlemi kullanmaktır.

12. Temel İşlemle Saymak ve Forward Substitution

Temel işlem sayma yöntemi (basic operation counting method) şunu sorar: Bu algoritmada kesin olarak yapılacak işlem hangisidir? Bu döngüde karşılaştırma, toplama, artırma ve atama hepsi temel işlem gibi görünür. Bunlardan biri seçilir. Döngünün kaç defa döneceği soruluyorsa karşılaştırmayla da hesap yapılabilir, ama sonra bir eksiltmek gerekir. Artırmayla veya atamayla da yapılabilir. Burada soru `T`'nin son değeri olduğu için temel işlem toplama (`+`) alınır.

Diğer katsayıları işin içine katarsak $3n + 4$ gibi gereksiz ayrıntılar çıkar. Hoca bunu temel işlem analizinin önemi olarak gösterdi. Gelecek derste bir sıralama algoritmasında, recursive konusunda da merge sort'ta bu analizin algoritmanın tamamını anlamaya yettiği görülecektir.

Toplam sembolü her zaman hemen akla gelmeyebilir. O zaman forward substitution yöntemi kullanılır: Değişkenin ilk birkaç iterasyondaki değerleri yazılır, desen (pattern) görülür ve sonra toplam veya çarpım sembolüyle ifade edilir. Bu biraz tahmine dayalı bir analizdir. Hoca, bu tarz sorularda desenin görülebilir veya bilinen formüllere uydurulabilir olduğunu söyledi. Çok karmaşık fonksiyonlar içeren sorularda bu mümkün olmaz.

Aşağıdaki kodların tümü derste $n = 10$ ile çalıştırıldı. Yalnızca döngünün gövdesi değişiyor.

12.1 T_1: `T = T + 1`

`T` başta 0'dır. Değerler 1, 2, 3, … diye gider. Her iterasyonda `T` bir artar, toplamda n tane terim gelir:

$$T_1(n) = \sum_{i=1}^{n} 1 = (n - 1 + 1) \cdot 1 = n$$

Toplam sembolünde üst sınır dahildir. Terim sayısı, son eksi ilk artı birdir ve toplanan şey `i`'ye bağlı olmadığı için sabittir. $n = 10$ için sonuç 10'dur.

12.2 T_2: `T = T + 2`

Bu kez `T` ikişer ikişer artar: 2, 4, 6, ... Temel işlem yine toplamadır:

$$T_2(n) = \sum_{i=1}^{n} 2 = 2n$$

$n = 10$ için sonuç 20'dir. Hoca buradan bir genelleme çıkardı: 2 yerine herhangi bir sabit a yazılsa sonuç $a \cdot n$ olur, yani doğrusal bir fonksiyon.

12.3 T_3: `T = T + i`

`i` kendi içinde artan bir değişkendir. Forward substitution yapalım. `T` başta 0'dır:

- Birinci iterasyon: 1
- İkinci iterasyon: $1 + 2$
- Üçüncü iterasyon: $1 + 2 + 3$
- n'inci iterasyon: $1 + 2 + \cdots + n$

Yani `T`, `i`'ye göre değişir:

$$T_3(n) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \frac{n^2 + n}{2}$$

$n = 10$ için $\frac{10 \cdot 11}{2} = 55$ olur ve program 55 yazar. Hoca terim toplamı formüllerinin listesini gelecek derste vereceğini söyledi.

Hoca bir de `i`'nin 0'dan başlatılmasını sordu. Matematiksel sonuç değişmez, çünkü 0 eklemek toplamı etkilemez. Ama algoritma bir iterasyon fazla çalışır.

Karmaşıklık sorusu.

Burada dersin en önemli karışıklığı çıktı. Hoca önce bunun karmaşıklığını sorup n^2 yanıtına yöneldi, sonra soruyu yanlış sorduğunu söyleyip düzeltti. Doğru ayrım şudur:

- $\frac{n^2+n}{2}$, bu kodda **`T`'nin ulaştığı son değerdir**, karmaşıklık analizinin sonucu değildir.
- Eğer bir X algoritmasının karmaşıklık analizi sonucu $T_X(n) = \frac{n^2+n}{2}$ çıksaydı, en büyük dereceli terimi alıp karmaşıklığa n^2 derdik.
- T_3 kodunun karmaşıklığı ise n'dir. Çünkü döngü n defa döner ve her turda bir birimlik bir toplama yapılır: $\sum_{i=1}^{n} 1 = n$. Toplamanın maliyetine `i` değil, bir birim deriz.

Derste 3 yanıtı da geldi. Hoca, 3 denebilmesi için toplamın formülle, hiç döngü kullanmadan yapılması gerektiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Hocanın "3" için söylediği, büyük olasılıkla $\frac{n(n+1)}{2}$ formülündeki işlem sayısıdır: bir toplama, bir çarpma ve bir bölme. Bu durumda işlem sayısı n'e bağlı olmaz, yani sabit olur.

Soru: Bu n^2, sağdaki ifadenin hesaplanmasının maliyeti mi?

Cevap: Hayır. Şöyle bir fonksiyon bulduğumuzu düşünün: Bu ifade bir algoritmanın karmaşıklık sonucu olsaydı n^2 olurdu. Oysa buradaki ifade `T`'nin değerini gösterir, bu kod için karmaşıklık n'dir.

Hocanın vurgusu: Karmaşıklık başka şeydir, T'nin ulaşacağı son değer başka şeydir.

12.4 T_4: `T = T + 2*i`

Forward substitution ile 2, 2 + 4, 2 + 4 + 6, ... gelir. Toplanan terim $2i$'dir. 2 sabit olduğu için dışarı çıkar:

$$T_4(n) = \sum_{i=1}^{n} 2i = 2\sum_{i=1}^{n} i = 2 \cdot \frac{n(n+1)}{2} = n(n+1)$$

Yani klavyeden girilen n değeri ile bir sonraki değerin çarpımı bulunur. $n = 10$ için $10 \cdot 11 = 110$ olur.

12.5 T_5: `T = T + T`

Hoca önce T'nin ilk değerini düzeltti. T başta 0 olursa $0 + 0 = 0$ olur ve hep 0 kalır. Bu yüzden `T = 1` verilir.

İterasyon	T
Başlangıç	1
1	$2 = 2^1$
2	$4 = 2^2$
3	$8 = 2^3$
4	$16 = 2^4$
n	2^n

T her defasında iki katına çıkar. Artık bir toplam sembolü yoktur, çarpım sembolü gibi çalışır. 2'ler n defa çarpılır:

$$T_5(n) = \prod_{i=1}^{n} 2 = 2^n$$

$n = 10$ için $2^{10} = 1024$ olur ve program 1024 yazar.

Karmaşıklık sorusu burada da aynıdır. T'nin son değeri 2^n'dir, ama bu kodun karmaşıklığı n'dir: n defa bir toplama yapılır, sonucun ne olduğuna bakılmaz.

Fakat bir algoritmanın karmaşıklık sonucu 2^n çıksaydı karmaşıklığı 2^n'lik olurdu. Hocanın böyle bir örneği şudur: Elinizde n bit var. Bu bitlerle 2^n farklı şifre üretilebilir. Alt küme problemi de hocanın bu gruptan andığı problemdir. Bu tür algoritmalar polinom zamanlı değil, üstel (exponential) zamanlıdır. Hocanın ifadesiyle bu çok kötü bir algoritmadır, çünkü problem zordur.

13. İç İçe Döngüler

Forward substitution iç içe döngülerde zorlaşır, çünkü hem `i`'ye hem `j`'ye göre değişim vardır. Bu yüzden doğrudan toplam sembolü yazılır. Dış döngü `i` için 1'den n'e, iç döngü `j` için 1'den n^2'ye gider.

13.1 T_6: `T = T + 1`

Hoca burada da önceki koddan kalan `T = T + T` ve `T = 1`'i düzeltti. T 0'dan başlar ve bir artar.

$$T_6(n) = \sum_{i=1}^{n} \sum_{j=1}^{n^2} 1 = \sum_{i=1}^{n} n^2 = n \cdot n^2 = n^3$$

İç kısmın terim sayısı $n^2 - 1 + 1 = n^2$'dir. Bu n^2, dışarıda n defa gerçekleşir. $n = 10$ için 1000 çıkar.

13.2 T_7: `T = T + i`

Artışlar artık `i` kadardır. İç döngü `j`'ye bağlıdır, dolayısıyla iç toplamda `i` sabittir:

$$T_7(n) = \sum_{i=1}^{n} \sum_{j=1}^{n^2} i = \sum_{i=1}^{n} n^2 \cdot i = n^2 \sum_{i=1}^{n} i = n^2 \cdot \frac{n(n+1)}{2} = \frac{n^3(n+1)}{2}$$

Dış toplamda n^2 sabit olduğu için dışarı çıkar, kalan $\sum i$ daha önce bulunan formüldür. $n = 10$ için $\frac{1000 \cdot 11}{2} = 5500$ olur.

13.3 T_8: `T = T + j`

Artışlar `j` kadardır. İç kısım 1'den n^2'ye sayıların toplamıdır. Terim toplamı formülünde son terim n^2'dir:

$$\sum_{j=1}^{n^2} j = \frac{n^2(n^2+1)}{2}$$

Bu ifade `i`'ye göre değişmez, sabittir. Dışarı atılınca içeride birler kalır ve 1'den n'e birlerin toplamı n'dir:

$$T_8(n) = \sum_{i=1}^{n} \frac{n^2(n^2+1)}{2} = n \cdot \frac{n^2(n^2+1)}{2} = \frac{n^3(n^2+1)}{2}$$

$n = 10$ için $\frac{1000 \cdot 101}{2} = 500 \cdot 101 = 50500$ olur.

> **Ders sonrası not:** Hoca işlemi söylerken "500 çarpı 101 bölü 2" dedi. Bölme zaten yapılmıştır, doğrusu $500 \cdot 101 = 50500$'dür. Hocanın okuduğu sonuç doğrudur (python ile doğrulandı).

13.4 Sekiz Kodun Bir Arada Çalıştırılması

Derste kodlar tek tek değiştirilerek çalıştırıldı. Aşağıdaki program hepsini bir araya getirir (`gcc -Wall -std=c89 -pedantic` ile uyarısız derlendi):

```c
#include <stdio.h>
#define N 10

int main(void)
{
    int i, j;
    long T;

    T = 0;
    for (i = 1; i <= N; i++)
        T = T + 1;
    printf("T1 = %ld\n", T);
```

```
    T = 0;
    for (i = 1; i <= N; i++)
        T = T + 2;
    printf("T2 = %ld\n", T);

    T = 0;
    for (i = 1; i <= N; i++)
        T = T + i;
    printf("T3 = %ld\n", T);

    T = 0;
    for (i = 1; i <= N; i++)
        T = T + 2 * i;
    printf("T4 = %ld\n", T);

    T = 1;
    for (i = 1; i <= N; i++)
        T = T + T;
    printf("T5 = %ld\n", T);

    T = 0;
    for (i = 1; i <= N; i++)
        for (j = 1; j <= N * N; j++)
            T = T + 1;
    printf("T6 = %ld\n", T);

    T = 0;
    for (i = 1; i <= N; i++)
        for (j = 1; j <= N * N; j++)
            T = T + i;
    printf("T7 = %ld\n", T);

    T = 0;
    for (i = 1; i <= N; i++)
        for (j = 1; j <= N * N; j++)
            T = T + j;
    printf("T8 = %ld\n", T);

    return 0;
}
```

Çıktı sırayla 10, 20, 55, 110, 1024, 1000, 5500, 50500'dür.

14. Son Değer ile Karmaşıklık Farklı Şeylerdir

Hoca T_6, T_7 ve T_8 için de aynı soruyu sordu. Gövdede `T + 1`, `T + i` veya `T + j` olması fark etmez. Temel işlem toplamadır ve her turda bir kez yapılır. Bakacağımız şey iç içe iki döngünün kaç defa döndüğüdür: İç döngüden n^2, dış döngüden n gelir, toplam n^3'lük bir karmaşıklık vardır. Bu, üçüncü dereceden polinom karmaşıklığına sahip bir algoritmadır.

Derste analiz edilen bütün kodları bir arada görelim:

Kod	T'nin son değeri	$n = 10$	Karmaşıklık
T_1: `T + 1`	n	10	n
T_2: `T + 2`	$2n$	20	n
T_3: `T + i`	$\frac{n(n+1)}{2}$	55	n
T_4: `T + 2*i`	$n(n+1)$	110	n
T_5: `T + T`	2^n	1024	n

Kod	T'nin son değeri	$n = 10$	Karmaşıklık
T_6: iç içe, `T + 1`	n^3	1000	n^3
T_7: iç içe, `T + i`	$\frac{n^3(n+1)}{2}$	5500	n^3
T_8: iç içe, `T + j`	$\frac{n^3(n^2+1)}{2}$	50500	n^3

Gelecek derste benzer sorular daha da karmaşıklaşacaktır: `for` döngülerinin sınırları değişecek, içine `if` eklenecek veya `while` döngüleriyle forward substitution yapılarak `T`'nin değeri tahmin edilecektir.

15. Ders Sonrası Kod Soruları

Ders bittikten sonra kalan öğrencilerle kod soruları konuşuldu.

15.1 Diziye Toplu İlk Değer

Soru: Bütün diziyi -1 yapmak için `for` yerine `int A[100] = {-1};` denedim. Olmadı. Neden?

Cevap: İlk eleman -1 olur, kalanlar 0 olur. İlk değer verme listesindeki değerler sırayla atanır, geri kalan elemanlar sıfır olur. Örneğin 100 elemanlı bir dizinin ilk 5'ini verirseniz kalanı otomatik 0 olur. İlk değer verilmiş ve verilmemiş değişkenler bellekte farklı şekilde saklanır.

Soru: Bunu yapmanın başka bir yolu yok mu?

Cevap: Yok. C'de değerleri varsayılan olarak atamak gibi bir şey yoktur, ancak döngüyle olur. Python gibi dillerde bu vardır, ama o da arka planda bir döngü gibi bir şeyle veya belki paralel işlemeyle bu işi yapar.

Soru: Bu, dizinin ilk elemanının adresiyle dizinin adresi aynı olduğu için mi?

Cevap: Evet, süslü parantez içinde aslında bir dizi gibi bir şey oluşturulur. İlk elemandan itibaren oraya kopyalanır.

> **Ders sonrası not:** Hoca döngüden başka yol olmadığını söyledi. `int` dizisi ve -1 için büyük olasılıkla `memset(A, -1, sizeof(A));` (`<string.h>`) de çalışır, çünkü -1'in bütün baytları `0xFF`'tir (gcc ile doğrulandı). Bu yöntem büyük olasılıkla yalnızca 0 ve -1 için böyle sonuç verir. Kalan elemanların 0 olması da büyük olasılıkla adreslerin aynılığından değil, dilin ilk değer verme kuralından gelir. Bu derste döngüyle yazmak en açık yoldur.

15.2 Dizinin Dışına Erişim

Bir öğrenci, sayım dizisi (`count`) kullanan algoritmayı yazmış ve program çökmüştü. Dizi 4 elemanlıydı, elemanlarının değerleri arasında 23, 27 ve 5555 vardı. `count` dizisi n'e göre, yani 4 elemanlı açılmıştı. Oysa kod `count[5555]`'e erişmeye çalışıyordu. Hoca bunun dizinin dışına erişim olduğunu, "boundary exception" denilen bir hata olduğunu ve işletim sisteminin bunu engellediğini anlattı.

Bu yüzden bazı algoritmalar dinamik bellek tahsisi bilinmeden yazılamaz. Hocanın daha önce "yapmayın" dediği şey buydu.

> **Ders sonrası not:** C büyük olasılıkla dizi sınırlarını kontrol etmez ve bir istisna (exception) mekanizması da yoktur. Program yalnızca erişilen adres, işletim sisteminin o prosese vermediği bir bellek bölgesine düşerse çöker. Sınırın biraz dışına yazmak büyük olasılıkla hata vermeden başka değişkenleri bozar. Hocanın uyarısı bu yüzden çökme olmasa da geçerlidir.

15.3 Yeniden Tanımlama ve Ortada Dizi Tanımı

Bir başka öğrencinin derleyicisi bir değişkenin tekrar tanımlandığını söylüyordu. Öğrenci 23. satırda arada bir dizi tanımlamıştı. Yukarıda tanımlanmış bir şey aşağıda tekrar tanımlanamaz.

Tanım da oraya yazılmaz, en yukarı alınmalıdır. Öğrencinin amacı önce maksimumu bulup diziyi ona göre açmaktı. Ama maksimum o noktada bilinmez. Bu da dinamik bellek konusudur.

Hocanın şimdilik önerisi, iki diziye de büyük bir boyut (örneğin 500) verip maksimuma kadarki kısmını kullanmaktır. Bunun çözümü vardır, ama bu dönemin konusu değildir.

Hoca burada algoritma ile gerçekleştirme (implementation) arasındaki farkı anlattı. Bir şey akış diyagramında doğru gösterilir, ama koda geçince hata alınır. Öğrenci "algoritma mı hatalı?" diye sorar. Algoritma hatalı değildir, dile özgü kuralları bilmek gerekir. Hocanın benzetmesi sıfat cümlesidir: Sıfat cümlesi her dilde ismi niteleyen aynı şeydir, ama Türkçe, İngilizce, Arapça ve Almanca'da farklı kurulur. Türkçe'de sıfat önde gelir, İngilizce'de bazı yapılar sona gelir.

15.4 `#define` Değişken Değildir

Soru: `int a[MAX]` yazarken `MAX`'i `#define` ile tanımladık. Bunu değişken gibi düşünüp öyle tanımlayamıyoruz, değil mi?

Cevap: `#define` ile yapılan şeyler değişken değildir, yerine koymadır (substitution). Program çalıştırılabilir hale gelmeden önce, derleme aşamasında metin olarak yerine konur. Yani aşağıda `100` yazılmış gibi olur. Amaç aynı şeyi tekrar tekrar yazmamaktır. `A` ve `B`'ye aynı değer verilecekse ve bundan eminseniz ikisine gerek de yoktur. Matrislerde farklı verilebilir. Bunlar sabittir. Hoca adım adım derlemeyi anlattığı dersi hatırlattı. Orada ara kodlarda bu görülür.

15.5 Değişkenlerin Yeri ve Diğer Diller

Soru: Başka kaynaklarda değişkenlerin fonksiyonun ortasında tanımlandığını gördüm. Standarda uygun mu?

Cevap: Uygun değildir, yalnızca üstte tanımlanır. Pascal'da program başlamadan önce yazılan `var` bölümü gibi, C'de de buna uyulmalıdır. Uyulmazsa farklı bir derleyicide, özellikle kontrolde kullanılan derleyicilerde hata alınır. Her fonksiyonun başında bütün değişkenler tanımlanır.

Soru: Swift'te oluyordu. Dil tabanı aynı olduğu için C'de de olur diye düşündüm.

Cevap: Olmaz. Hoca yine dillerle benzetti: İngilizce ve Almanca ikisi de German dili olsa da birinde olan öbüründe olmayabilir. Az önce yapılan şey C# ve Java'da olur, C'de olmaz.

Hoca C'yi burada "sinsi bir dil" olarak niteledi. Bazı derleyiciler derlerken hata vermez, program çalışırken hata alırsınız. Derleme zamanı hatasını programı çalıştırmadan düzeltme imkânınız vardır. Çalışma zamanında oluşan hata çok daha maliyetlidir.

15.6 Matrisin Transpozu ve 90 Derece Döndürme

Soru: Matrisi 90 derece çevirmek zaten transpozunu almak olmuyor mu?

Cevap: Hayır. Saat yönünde veya tersinde çevrilebilir. Transpoz yalnızca satır ve sütunların kendi arasında yer değiştirmesidir. Bir resmi 90 veya 180 derece döndürmek başka bir şeydir. Hoca dört farklı dönüşüm olduğunu söyledi: 90, 180, 270, 360 derece.

Öğrenci, saat yönünün tersine 90 derecenin saat yönünde 270 derece demek olduğunu söyledi. Hoca bunu onayladı, ama transpozun başka bir şey olduğunu tekrarladı.

> **Ders sonrası not:** Hoca saat yönünün tersine çevirmenin transpoz gibi düşünülebileceğini ve transpozun bu dönüşümlerden biri olduğunu söyledi. Doğrusu büyük olasılıkla, transpozun döndürme olmadığıdır: Transpoz ana köşegene göre yansımadır. Saat yönünün tersine 90 derece döndürme, transpoz alınıp satırların sırasının ters çevrilmesidir. Saat yönünde 90 derece ise transpoz alınıp her satırın kendi içinde ters çevrilmesidir (python ile doğrulandı).

15.7 Matrisi Genişletmek

Soru: Bir soruda 3×4'lük matrisin satır toplamlarının son sütuna eklenip 4×4'lük hale getirilmesi isteniyor. Kodda matrisi nasıl genişletirim?

Cevap: Şu anki bilgilerle genişletemezsiniz, dinamik bellek gerekir. Matrisi baştan bir boyut fazla tanımlayın. 3×3'lükse 4×4'lük tanımlayın.

15.8 Sonraki Dönemle Bağlantı

Hoca, dinamik belleği bu derste de anlatabileceğini, ama programın diğer gruplarla birlikte yürüdüğünü söyledi. Yapısal Programlamaya Giriş'in kapsamı fonksiyonlar, diziler ve biraz pointer'dır. Gelecek dönemin Yapısal Programlama dersi, C'nin sözdizimi kuralları, özel dönüşümler, döngüler, diziler ve `if` üzerine kısa bir tekrarla başlar. Sonra pointer ve dinamik bellekle yeni konulara girilir. Ardından structure ve dosya işlemleri gelir. Hocaya göre structure mantığı anlaşılınca nesneye yönelik mantık ve Java'nın mantığı da büyük ölçüde çözülür.

Bu dönem ise algoritmik problemleri kağıt üzerinde, akış diyagramıyla nasıl çözeceğinizi düşünmek için bir fırsattır. Çünkü her şey doğrudan kodlanamayabilir, dinamik matris ve dinamik dizi gerektiren şeyler olabilir.

16. Dil Öğrenmek ve Öğrenmeyi Öğrenmek

Hoca dersin sonunda öğrenme üzerine konuştu. Laboratuvar çalışmasında bir öğrencinin noktalı virgül, `if` veya `for` hatasını anında uyarmak öğrenmeye yol açar. Ama evde kod yazmayan öğrenci bir hafta sonra aynı hatayı yine yapar.

Hocaya göre üniversitede artık "öğrenmeyi öğrenme" vardır. Derste anlatılan şey aslında bir dilin ve kavramların nasıl öğrenileceğidir. Bir yerden sonra sınav sorularının tarzına uygun bir sürü test çözmek ve onları ezberlemek hayatta karşılığı olan bir şey değildir. Hoca kendi öğrencilik yıllarından örnek verdi: Biyolojiden sekiz kitap bitirmişti, ama genetik bölümünde okunacak şeylerle arasında dağlar kadar fark vardı. Geometride de soru çözdükçe öğrendiğini zannetmişti, oysa "böyle bir soru gelirse bu yöntemi kullanırım" diye ezberliyordu.

Hoca derin öğrenmeyi de buna benzetti: Yapay zekada örnekler üzerinden öğrenme vardır. Derin öğrenme, örnek sayısını bir milyon, on milyon görüntüye kadar artırır. Hocaya göre bu, aslında "öğrenmesin, ezberlesin" demektir. Başarılı sonuçlar verir, ama başarısız sonuçlara da bakmak gerekir.

> **Ders sonrası not:** Bu, hocanın sınav hazırlığı için kurduğu bir benzetmedir. Hoca derin öğrenmeye bir tür ezber dedi. Makine öğrenmesinde hedef büyük olasılıkla ezber değil, görülmemiş örneklere genellemedir. Ezberleme ayrı bir sorun olarak (aşırı öğrenme) ele alınır.
>
> **Hocanın vurgusu:** Bizim bölümümüz ve teknolojik bölümler soyut düşünmeyi, yeniliklere açık olmayı ve kendi kendine öğrenmeyi gerektirir. Her zaman karşınızda bir öğretici olmayacak.

Dersin Toparlaması

Bu derste iki ana çizgi birleşti. Birincisi C'nin parametre aktarımındaki sessizliğidir: Call by value ve call by reference tekrar edildi, ardından yanlış tipte gönderilen değerlerin derleme veya çalışma hatası vermeden dönüştürüldüğü görüldü. `f('A')` 65 yazar, `h(256)` 0 yazar, string'in adresi tam sayı gibi basılır. Java aynı durumların çoğunu derleme zamanında yakalar. Doğrusal sıralamanın sorunları için de frekans saklama ve değer-frekans tablosu önerildi, bu çözümler bellek-zaman ödünleşimini gösterdi.

İkinci çizgi iteratif algoritmaların analizidir. Temel işlem seçilir, iterasyon sayısı toplam veya çarpım sembolüyle yazılır, gerekirse forward substitution ile desen bulunur. T_1'den T_8'e sekiz kod bu yöntemle çözüldü ve her biri $n = 10$ ile doğrulandı. En önemli sonuç, `T`'nin ulaştığı son değer ile kodun karmaşıklığının farklı şeyler olmasıdır. Gelecek derste `for` sınırlarının değiştiği, `if` ve `while` içeren döngüler ile selection sort ve bubble sort gibi bir sıralama algoritmasının analizi işlenecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.